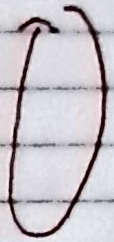
Design and Analysis of Software Systems

Development Cycle:- Dev Life Cycle [SDLC]
Software

↳ Determine whether the proposed sys is viable

(i) Feasibility Analysis:- Req for proposal
(ii) Requirements / Specifications:- What is needed, ~~How to~~
(iii) Design:- How to do
(iv) Implementation:- Modelling
(v) Testing:- Static and Dynamic & Deployment
(vi) Main Maintenance:-



Greenfield & Brownfield systems:-
↓ ↓
Develop from scratch | Work on existing systems

Maintenance is max effort

Modules are never integrated in one shot, multiple steps. During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model:- Types of SDLC, SDLC models
(i) Traditional → Plan driven
(ii) Agile → Dynamic

Processes must continually improve

Debriefing - Planning & Execution

Convergence:- Estimates get more accurate as the rounds

Design and Analysis of Software Systems

Development Cycle :- Dev life Cycle [SDLC]

↳ Determine whether the proposed sys is viable

(i) Feasibility Analysis :- Req for proposal

(ii) Requirements / Specifications :- What is needed, ~~How to do~~

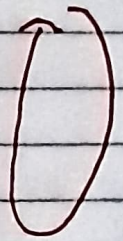
(iii) Design :- How to do

Modelling

(iv) Implementation :-

(v) Testing :- Static and Dynamic & Deployment

(vi) ~~Main~~ Maintenance :-



Greenfield & Brownfield systems :-

↓ ↓

Develop from scratch — Work on existing systems

Maintenance is max effort

Modules are never integrated in one shot, multiple steps. During each step, the partially integrated system is tested

Stuff keeps changing, products must evolve

Process Model :- Types of SDLC, SDLC models

(i) Traditional → Plan driven

(ii) Agile → Dynamic

Processes must continually improve

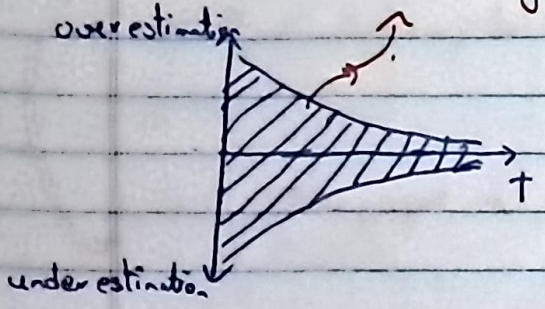
Debriefing - Planning & Execution

Convergence :- Estimates get more accurate as the rounds

progressed

Upfront Planning:-

Cone of Uncertainity



Traditional:- No iterations, one planning session and no changes

Agile:- Many iterations, multiple shorter planning sessions between iterations

Team Dynamics & Self-organization

Leadership and Psychological Safety

Lean Principles

Work in Progress and The Definition of Done
when is the unit
actually done

Rework:- Starting from scratch once again
results
Technical Debt:-

PDCA – Continuous improvement mindset

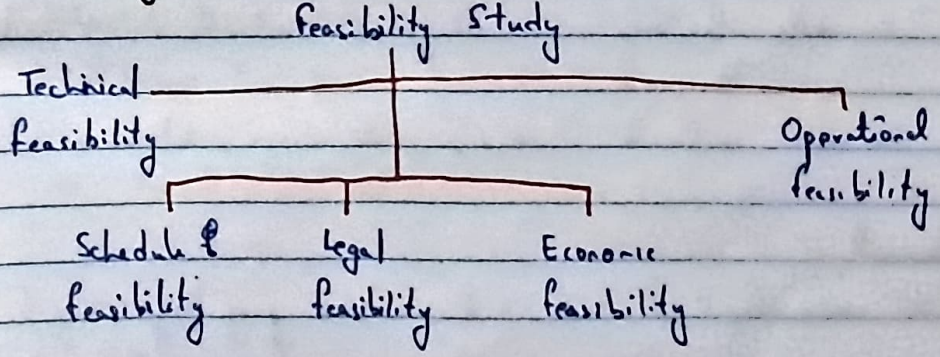
Alexander Deming

Plan – Do – Check – Act

Plan	Do	Check	Act
Identifying Problems / oppurtunities, define objectives and plan changes Planning Estimation Requirements Design	Implement on a small scale Implementation	Measure results and compare against expectations Testing Deployment	Standardise success or adjust & iterate Retrospect [Maintenance, Documentation, Changes]

1 PDCA cycle ≈ 1 iteration

Feasibility:-



Determine whether the proposed system is viable & worth pursuing

Requirements Engineering:-
Elicit, analyse, specify, validate & manage stakeholder needs so that the system built meets the intended purpose & constraints

SRS – Software Requirement Specifications

PRD – Product Requirement Document

Consists of

- ↳ req gathering & analysis (from diff povs)
- ↳ req specification
- ↳ verification & validation
- ↳ req management & change control

Design :-

Transform reqs into a robust, maintainable and implementable architecture & detailed designs that guide dev & testing.

↓

High level design (Arch) → ADD (Arch desc doc)

- ↳ decompose the system into modules / components
- ↳ represent invocation relationships among modules / components
- ↳ specify constraints

Detailed Design → DD (Design Doc)

- ↳ UML
- ↳ data structures, class diagrams, interfaces & modules are designed
- ↳ API contracts, seqs, state diags, database schema.

Implement :-

- ↳ code front & back end in some prog lang

Testing :-

Test plan ---> Implementation

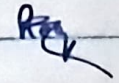
↓ !---> Design

Test !--> RE

Oracle

Dilbert!
PhD Comics!

V-Model



Maintenance :-

- Prevention → Defection prevention
- Correction → Bug fixes
- Perfection → improvements / enhancements
- Adaptive → Port software to new environments

Process Models :-

Traditional :- (plan driven)

- Waterfall model
- Prototyping
- Evolutionary
- Spiral model etal

Agile :-

- eXtreme Programming (XP)
- Scrum
- Crystal
- Feature Driven Development (FDD)

Water fall model :- → Stability !

Seq process

Req. design, implementation, testing & maintenance need to be followed in that order

Each phase needs to be completed before moving on.

Challenges :-
- ↳ Heavy weight proc
- ↳ Doc intensive
- ↳ Minimal consumer involvement after req phase
- ↳ Less flexible design
- ↳ Big bang approach to coding & integration
- ↳ One shot delivery opportunity
- ↳ Limited opp for proc improvement

Good for systems with stable requirements

Prototyping Model → Clarity !

Before starting actual dev, working prototypes are built to elicit custome. feedback.

toy model ↵
limited perf & efficiency.

- ↳ Throwaway
- ↳ Evolutionary.

Good for sys with under req, & with focus on UI/UX

Morning!

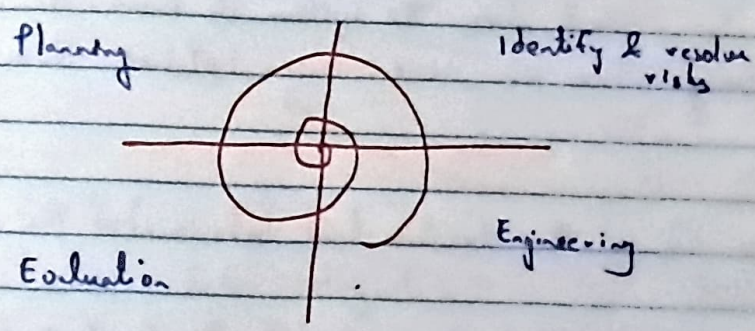
Evolutionary process model → Speed!

Sys is broken down into several modules which can be incrementally implemented & delivered.

Iterative incremental model → Model Agile

Spiral Model → Safety!

Strong emphasis on risk.
Cont. cust feedback
· Suitable for large, complex high risk systems
Expensive but highly controlled.



Each model as each loop of the spiral might be a process by itself.

Agile :-

1990 - 2000

→ Close collaboration blw dev & business experts
• 12 practices in the manifesto.
→ Maximise face-to-face communication (keep customer in the loop always)
→ Self organising teams, everyone knows what they are doing and are owners themselves

Many models under Agile

Incremental & Iterative in nature

Characteristics:-

→ Short, but many quick releases
→ Planning based on user stories
→ Each iteration deals with all life cycle activities
→ Testing ⇒ unit testing for deliverables; acceptance tests for each release
→ Flexible Design ⇒ evolution vs big upfront effort
→ Reflection after each cycle
→ Several technical & customer focused presentation opportunities

User Stories:- What does the system do from the user's pov? ~~Many~~ One or more user stories form a feature

Refactoring:- Change the structure (not behaviour) of the code

Dead Code:- Code that is very difficult to test / not possible to test at times

Open-Closed Principle:- Code is open only for extension, closed for modification

SOLID principles for OOP

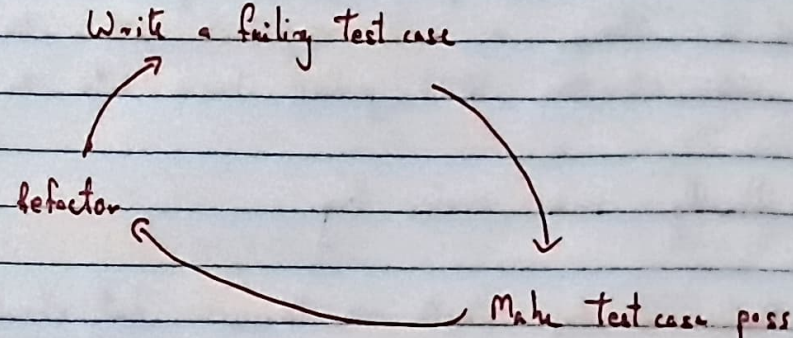
Results in Technical Debt → debt incurred cause of bad coding practices.

Code Smell ← Bad code (Ugly practices)

Long methods violate single responsibility principle
↳ Modularize

Test Driven Development:-

Write Tests First!
Test drives the development process
Write and Run testcases.
Then write code to ensure testcases pass.



Source Code Integration

Big Bang Integration is bad

Source code (download from version control)
Resolve dependencies
Compile & Run
Manual Testing
Package & Deploy.

Continuous Integration → ~~Devops~~ Development practice where devs merge changes into a shared repository frequently.

Commit → Build → Test → Feedback

Automated ↻ → Occurs continuously

Build Tools help with build process (MAVEN, ANT)

CD → Continuous Deployment (Automated)

↳ Continuous Delivery (Manual)

SCRUM Process:-

Backlog of User Stories

SCRUM master splits the backlog into multiple sprints.

If a story is not completed within the sprint, it is pushed back into the backlog.

Usually 2-4 weeks long.

Daily Standup Meets → Discuss what should be done, what went well, etc to stay updated